

Untitled

Ana Julia Sanches de Araujo

2026-09-17

```
# Carregue os pacotes necessários
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.2

-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(leaflet)
```

```
airports <- read_csv("airports.csv")
```

Rows: 322 Columns: 7

-- Column specification -----

Delimiter: ","

chr (5): IATA_CODE, AIRPORT, CITY, STATE, COUNTRY

dbl (2): LATITUDE, LONGITUDE

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```

# 1. Definindo a função com os argumentos exigidos
analisa_aeronave <- function(tail_number, arquivo) {

  # =====
  # PARTE A: LEITURA EM LOTES E FILTRAGEM
  # =====

  # Cria a "receita" para filtrar apenas o avião desejado em cada lote
  filtra_aviao <- function(df, pos) {
    df %>%
      filter(TAIL_NUMBER == tail_number)
  }

  # Executa a leitura do arquivo grande por partes (chunks)
  voos_filtrados <- read_csv_chunked(
    file = arquivo,
    chunk_size = 1000000,
    callback = DataFrameCallback$new(filtra_aviao)
  )

  # =====
  # PARTE B: ORDENAÇÃO E TRATAMENTO DA TABELA
  # =====

  voos_tratados <- voos_filtrados %>%
    # Ordena os voos cronologicamente de maneira linear no tempo
    arrange(MONTH, DAY, SCHEDULED_DEPARTURE) %>%

    # Remove linhas com valores faltantes que podem bugar o mapa
    filter(!is.na(ORIGIN_AIRPORT) & !is.na(DESTINATION_AIRPORT)) %>%

    # Calcula uma estatística de interesse: Velocidade Média (Milhas por minuto)
    # Evita divisão por zero caso o AIR_TIME seja NA ou zero
    mutate(velocidade_media = ifelse(AIR_TIME > 0, DISTANCE / AIR_TIME, 0))

  # Adiciona as coordenadas exatas do aeroporto de ORIGEM
  voos_com_origem <- voos_tratados %>%
    left_join(airports, by = c("ORIGIN_AIRPORT" = "IATA_CODE")) %>%
    rename(lat_origem = LATITUDE, lon_origem = LONGITUDE)

  # Adiciona as coordenadas exatas do aeroporto de DESTINO
  tabela_final <- voos_com_origem %>%

```

```

left_join(airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE")) %>%
rename(lat_destino = LATITUDE, lon_destino = LONGITUDE)

# =====
# PARTE C: DESENHANDO O MAPA
# =====

# Inicia o mapa base
mapa_trajeto <- leaflet(data = tabela_final) %>%
  addTiles()

# O leaflet não desenha segmentos individuais com addPolylines facilmente de forma nativa.
# Para aplicar a espessura baseada na velocidade, adicionamos cada trajeto (linha por linha)
for (i in 1:nrow(tabela_final)) {
  linha <- tabela_final[i, ]

  # Só desenha se as coordenadas existirem
  if (!is.na(linha$lat_origem) && !is.na(linha$lat_destino)) {
    mapa_trajeto <- mapa_trajeto %>%
      addPolylines(
        lng = c(linha$lon_origem, linha$lon_destino),
        lat = c(linha$lat_origem, linha$lat_destino),

        # A espessura da linha (weight) reflete a velocidade média do voo
        weight = linha$velocidade_media / 2,

        # Estética básica para visualizar melhor
        color = "blue",
        opacity = 0.6,

        # Adiciona um balão interativo ao clicar na linha
        popup = paste("Origem:", linha$ORIGIN_AIRPORT,
                      "<br>Destino:", linha$DESTINATION_AIRPORT,
                      "<br>Data:", linha$DAY, "/", linha$MONTH,
                      "<br>Velocidade:", round(linha$velocidade_media, 2), "milhas/min")
      )
  }
}

# =====
# PARTE D: RETORNO DA FUNÇÃO
# =====

```

```

# Retorna uma lista nomeada conforme exigido no roteiro
resultado <- list(
  tabela = tabela_final,
  grafico = mapa_trajeto
)

return(resultado)
}

```

```

# Lembre-se de colocar o caminho correto onde o arquivo flights.csv está salvo
meu_teste <- analisa_aeronave(
  tail_number = "N431WN",
  arquivo = "flights.csv"
)

```

```

-- Column specification -----
cols(
  .default = col_double(),
  AIRLINE = col_character(),
  TAIL_NUMBER = col_character(),
  ORIGIN_AIRPORT = col_character(),
  DESTINATION_AIRPORT = col_character(),
  SCHEDULED_DEPARTURE = col_character(),
  DEPARTURE_TIME = col_character(),
  WHEELS_OFF = col_character(),
  WHEELS_ON = col_character(),
  SCHEDULED_ARRIVAL = col_character(),
  ARRIVAL_TIME = col_character(),
  CANCELLATION_REASON = col_character()
)
i Use `spec()` for the full column specifications.

```

```

# Para ver a tabela:
# View(meu_teste$tabela)

# Para ver o mapa:
# meu_teste$grafico

```

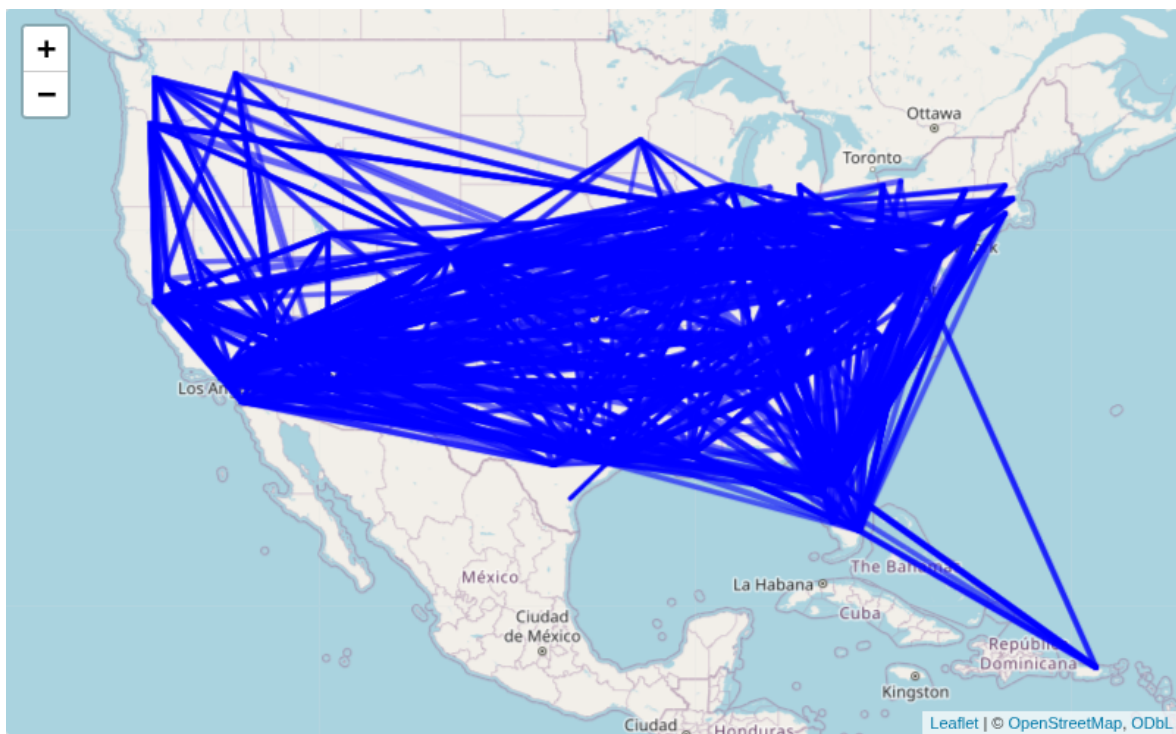


Figure 1: Trajeto do Voo

```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-09-16 23:00:44 -03"
```